

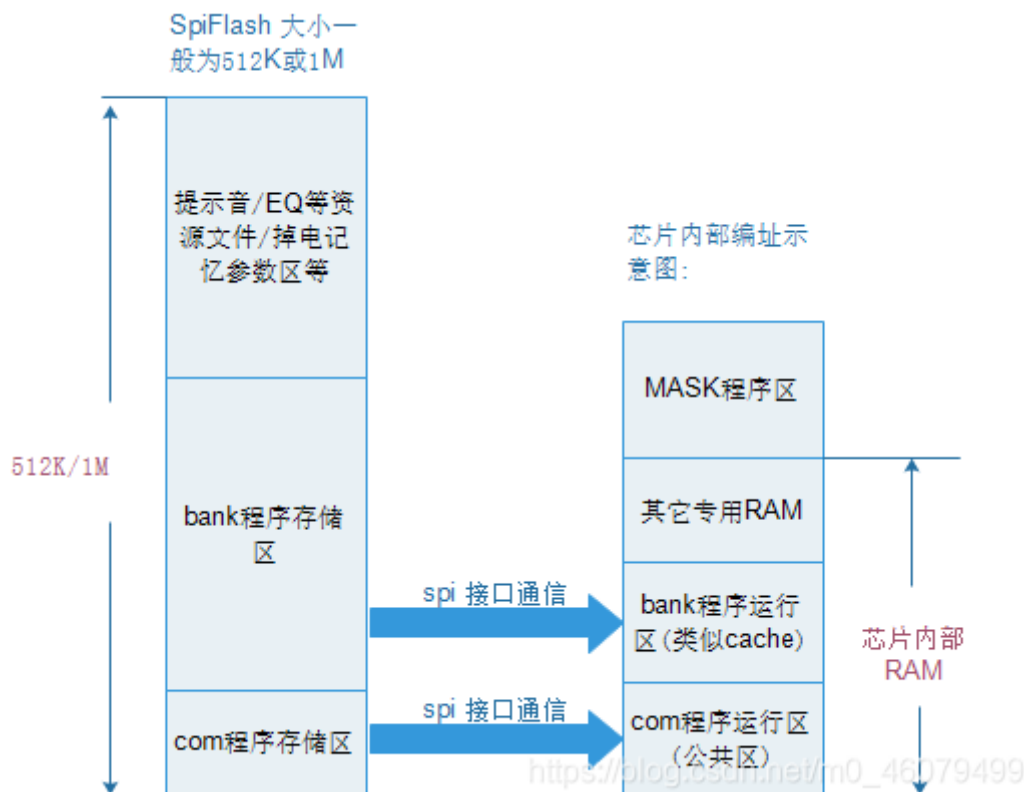
芯片框架简述

1、框架总述

随着蓝讯蓝牙方案在这一年两迅速崛起，公司也开始涉及到蓝讯蓝牙方案开发，在这里记录一些找到资料和自己的理解，和大家一起分享。

LX 蓝牙芯片采用最近比较流行的 RISC-V(32 位)开源内核架构 + 国产 RT-Thread 操作系统.不过从代码上来看，操作系统代码已经被封装到库中，一般用户可以用不涉及操作系统代码，降低了开发难度。

LX 芯片“冯·诺依曼结构”，即代码与数据的统一编址。框架结构大致如下：



芯片内部一般会封装一颗 512K 或 1M SpiFlash，用于存放代码及资源文件/参数记忆等. SpiFlash 和芯片之间通过 spi 接口进行通信。

首先,代码不会直接在 SpiFlash 上运行, SpiFlash 中所有程序及数据均需要先通过

spi 接口加载到芯片 RAM 中, CPU 再从 RAM 中取指令或数据运行.

2、com 区和 bank 区

基于上述的程序存储框架, LX 芯片在程序编写时, 有两个重要的概念: com 区(公共区) 和 bank 区.

com 区(公共区):

芯片上电, 一般从 Mask 程序区开始运行, 在进入 main 函数之前, 程序会先把 com 区程序从 Flash 加载到芯片内部 Ram. 由于在程序的整个生命周期内, com 区程序会一直保留在 RAM 中. CPU 执行 com 区代码会很快.

但由于芯片 RAM 有限, com 区一般分配在几十 K 以内.

bank 区(也称为 flash 区):

Flash 中的 bank 区(存储区)一般是几百 K 以上.

RAM 中的 bank 程序运行区(类似 cache), 一般从几 K 到几十 K 不等.

由于 RAM 中的 bank 区远远小于 FLASH 中的 bank 区, 所以 CPU 会根据需要不断把 Flash(bank 程序存储区)中的代码动态替换(加载)到 RAM 中的 bank 程序运行区运行. 由于芯片与 Flash 之间通过 spi 进行通信, bank 区代码执行速度相对比较慢.

3、开发时需要注意:

综上所述, 开发时需要注意如下:

1) 中断函数(及其子函数)必须放入 com 区, 否则(放入 bank 区)会导致死机.

中断响应需要非常及时, com 区程序常驻于 RAM 中, CPU 可以迅速响应中断函

数.

如果中断函数放入 bank 区, 中断响应时, 可能该 bank 区还未加载到 RAM 中, 还需要先加载再运行, 耗时相对较慢. 为了防止中断响应慢, 芯片做了限定: 中断中加载 bank 区代码则直接死机.

另外需要特别注意: 中断函数中不能有 **switch** 语句, switch 语句编译后生成的跳转常量表会默认放到 bank 区, 引起中断函数访问 bank 区死机. 请用 if-else 语句代替 switch 语句.

一些实时性要求比较高的代码, 首选放入 com 区.

2). 函数前没有 AT 指定存放段的代码, 默认放入 bank 区, bank 区代码自动加载(或替换), 不需要人工干预.

由于 bank 区加载到 RAM, 需要 spi 通信, IO 翻转会有一定干扰, 在一些对干扰敏感的应用中, 如 FM, 可以把所有相关程序放入同一个命名的 bank, 这样 bank 加载时, 会大概率地把所有同一 bank 中的程序一次性加载到 RAM 中, 减少程序运行时可能不断多次地加载引起的 spi 通信干扰.

4、函数放入 com 区的写法

在函数前面加入 AT(.指定段名到 com 区) 即可. 示例如 usr_tmr1ms_isr 函数前面的 AT(.com_text.timer)

```
AT(.com_text.timer)
void usr_tmr1ms_isr(void)
{
    gui_scan();
    .....
}
```

在 map.text 中可以看到, usr_tmr1ms_isr 位于 0x20e4a. 位于 0x20000~
(0x20000+34K) 的 com 内.

```
.com_text.timer
0x00000000000020e4a      0x10e Output\obj\platform\bsp\bsp_sys.o
0x00000000000020e4a      usr_tmr1ms_isr
0x00000000000020e5c      usr_tmr5ms_isr
0x00000000000020f3e      timer1_isr
```

另外有一点, printf 参数中的字符串常量也是默认放在 bank 区中, 如果在中断中调用 printf, 也需要把字符串常量放入 com 区.

如下:

```
AT(.com_text.str1)
const char str1[] = "Com String";
AT(.com_text.str2) // 注意每个字符串前都需要增加 AT 定位到 com 区去.
const char str2[] = "val = %d\n";
```

调用 printf 时, 直接使用该字符串即可:

```
AT(.com_text.timer)
void usr_tmr5ms_isr(void)
{
    printf(str1);
    printf(str2, test_val);
    .....
}
```